Games

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Sudoku Puzzles 3

2 Family Games America's Quantumino solver 19

3 Hexads in S(5, 6, 12) 31

4 Internals 41

5 Indices and Tables 43

Python Module Index 45

Index 47

Sage includes a sophisticated Sudoku solver. It also has a Rubik's cube solver (see Rubik's Cube Group).

CHAPTER

ONE

SUDOKU PUZZLES

This module provides algorithms to solve Sudoku puzzles, plus tools for inputting, converting and displaying various ways of writing a puzzle or its solution(s). Primarily this is accomplished with the *sage.games.sudoku.Sudoku* class, though the legacy top-level *sage.games.sudoku.sudoku()* function is also available.

AUTHORS:

- Tom Boothby (2008/05/02): Exact Cover, Dancing Links algorithm
- Robert Beezer (2009/05/29): Backtracking algorithm, Sudoku class

class sage.games.sudoku.**Sudoku**(*puzzle*, *verify_input=True*)

Bases: SageObject

An object representing a Sudoku puzzle. Primarily the purpose is to solve the puzzle, but conversions between formats are also provided.

INPUT:

- **puzzle – the first argument can take one of three forms**
  - list - a Python list with elements of the puzzle in row-major order, where a blank entry is a zero
  - matrix - a square Sage matrix over **Z**
  - string - a string where each character is an entry of the puzzle. For two-digit entries, a = 10, b = 11, etc.
- verify_input – boolean (default: True); use False if you know the input is valid

EXAMPLES:

```
sage: a = Sudoku('5...8..49...5...3..673....115.........2.8.........187....415..
↪3...2...49..5...3')
sage: print(a)
+-----+-----+-----+
|5    |  8  |  4 9|
|     |5    |  3  |
|  6 7|3    |    1|
+-----+-----+-----+
|1 5  |     |     |
|     |2   8|     |
|     |     |  1 8|
+-----+-----+-----+
|7    |    4|1 5  |
|  3  |    2|     |
|4 9  |  5  |    3|
```

(continues on next page)

(continued from previous page)

```
+-----+-----+-----+
sage: print(next(a.solve()))
+-----+-----+-----+
|5 1 3|6 8 7|2 4 9|
|8 4 9|5 2 1|6 3 7|
|2 6 7|3 4 9|5 8 1|
+-----+-----+-----+
|1 5 8|4 6 3|9 7 2|
|9 7 4|2 1 8|3 6 5|
|3 2 6|7 9 5|4 1 8|
+-----+-----+-----+
|7 8 2|9 3 4|1 5 6|
|6 3 5|1 7 2|8 9 4|
|4 9 1|8 5 6|7 2 3|
+-----+-----+-----+
```

```
>>> from sage.all import *
>>> a = Sudoku('5...8..49...5...3..673....115.........2.8.........187....415..3.
↪..2...49..5...3')
>>> print(a)
+-----+-----+-----+
|5    |  8  |  4 9|
|     |5    |  3  |
|  6 7|3    |    1|
+-----+-----+-----+
|1 5  |     |     |
|     |2   8|     |
|     |     |  1 8|
+-----+-----+-----+
|7    |    4|1 5  |
|  3  |    2|     |
|4 9  |  5  |    3|
+-----+-----+-----+
>>> print(next(a.solve()))
+-----+-----+-----+
|5 1 3|6 8 7|2 4 9|
|8 4 9|5 2 1|6 3 7|
|2 6 7|3 4 9|5 8 1|
+-----+-----+-----+
|1 5 8|4 6 3|9 7 2|
|9 7 4|2 1 8|3 6 5|
|3 2 6|7 9 5|4 1 8|
+-----+-----+-----+
|7 8 2|9 3 4|1 5 6|
|6 3 5|1 7 2|8 9 4|
|4 9 1|8 5 6|7 2 3|
+-----+-----+-----+
```

backtrack()

Return a generator which iterates through all solutions of a Sudoku puzzle.

This function is intended to be called from the *solve()* method when the `algorithm='backtrack'` option is specified. However it may be called directly as a method of an instance of a Sudoku puzzle.

At this point, this method calls *backtrack_all()* which constructs *all* of the solutions as a list. Then the present method just returns the items of the list one at a time. Once Cython supports closures and a yield statement is supported, then the contents of `backtrack_all()` may be subsumed into this method and the *sage.games.sudoku_backtrack* module can be removed.

This routine can have wildly variable performance, with a factor of 4000 observed between the fastest and slowest 9×9 examples tested. Examples designed to perform poorly for naive backtracking, will do poorly (such as `d` below). However, examples meant to be difficult for humans often do very well, with a factor of 5 improvement over the DLX algorithm.

Without dynamically allocating arrays in the Cython version, we have limited this function to 16×16 puzzles. Algorithmic details are in the *sage.games.sudoku_backtrack* module.

EXAMPLES:

This example was reported to be very difficult for human solvers. This algorithm works very fast on it, at about half the time of the DLX solver. [sudoku:escargot]

```
sage: g = Sudoku('1....7.9..3..2...8..96..5....53..9...1..8...26....4...3.....
↪.1..4......7..7...3..')
sage: print(g)
+-----+-----+-----+
|1    |    7|  9  |
|  3  |  2  |    8|
|    9|6    |5    |
+-----+-----+-----+
|    5|3    |9    |
|  1  |  8  |    2|
|6    |    4|     |
+-----+-----+-----+
|3    |     |  1  |
|  4  |     |    7|
|    7|     |3    |
+-----+-----+-----+
sage: print(next(g.solve(algorithm='backtrack')))
+-----+-----+-----+
|1 6 2|8 5 7|4 9 3|
|5 3 4|1 2 9|6 7 8|
|7 8 9|6 4 3|5 2 1|
+-----+-----+-----+
|4 7 5|3 1 2|9 8 6|
|9 1 3|5 8 6|7 4 2|
|6 2 8|7 9 4|1 3 5|
+-----+-----+-----+
|3 5 6|4 7 8|2 1 9|
|2 4 1|9 3 5|8 6 7|
|8 9 7|2 6 1|3 5 4|
+-----+-----+-----+
```

```
>>> from sage.all import *
>>> g = Sudoku('1....7.9..3..2...8..96..5....53..9...1..8...26....4...3.....
↪1..4......7..7...3..')
>>> print(g)
+-----+-----+-----+
|1    |    7|  9  |
```

(continues on next page)

(continued from previous page)

```
|  3  |  2  |    8|
|    9|6    |5    |
+-----+-----+-----+
|    5|3    |9    |
|  1  |  8  |    2|
|6    |    4|     |
+-----+-----+-----+
|3    |     |  1  |
|  4  |     |    7|
|    7|     |3    |
+-----+-----+-----+
>>> print(next(g.solve(algorithm='backtrack')))
+-----+-----+-----+
|1 6 2|8 5 7|4 9 3|
|5 3 4|1 2 9|6 7 8|
|7 8 9|6 4 3|5 2 1|
+-----+-----+-----+
|4 7 5|3 1 2|9 8 6|
|9 1 3|5 8 6|7 4 2|
|6 2 8|7 9 4|1 3 5|
+-----+-----+-----+
|3 5 6|4 7 8|2 1 9|
|2 4 1|9 3 5|8 6 7|
|8 9 7|2 6 1|3 5 4|
+-----+-----+-----+
```

This example has no entries in the top row and a half, and the top row of the solution is `987654321` and therefore a backtracking approach is slow, taking about 750 times as long as the DLX solver. [sudoku:wikipedia]

```
sage: c = Sudoku('.............3.85..1.2.......5.7.....4...1...9.......5.....
↪.73..2.1........4...9')
sage: print(c)
+-----+-----+-----+
|     |     |     |
|     |    3|  8 5|
|    1|  2  |     |
+-----+-----+-----+
|     |5   7|     |
|    4|     |1    |
|  9  |     |     |
+-----+-----+-----+
|5    |     |  7 3|
|    2|  1  |     |
|     |  4  |    9|
+-----+-----+-----+
sage: print(next(c.solve(algorithm='backtrack')))
+-----+-----+-----+
|9 8 7|6 5 4|3 2 1|
|2 4 6|1 7 3|9 8 5|
|3 5 1|9 2 8|7 4 6|
+-----+-----+-----+
|1 2 8|5 3 7|6 9 4|
```

(continues on next page)

(continued from previous page)

```
|6 3 4|8 9 2|1 5 7|
|7 9 5|4 6 1|8 3 2|
+-----+-----+-----+
|5 1 9|2 8 6|4 7 3|
|4 7 2|3 1 9|5 6 8|
|8 6 3|7 4 5|2 1 9|
+-----+-----+-----+
```

```
>>> from sage.all import *
>>> c = Sudoku('..............3.85..1.2.......5.7.....4...1...9.......5.....
↪73..2.1........4...9')
>>> print(c)
+-----+-----+-----+
|     |     |     |
|     |    3|  8 5|
|    1|  2  |     |
+-----+-----+-----+
|     |5   7|     |
|    4|     |1    |
|  9  |     |     |
+-----+-----+-----+
|5    |     |  7 3|
|    2|  1  |     |
|     |  4  |    9|
+-----+-----+-----+
>>> print(next(c.solve(algorithm='backtrack')))
+-----+-----+-----+
|9 8 7|6 5 4|3 2 1|
|2 4 6|1 7 3|9 8 5|
|3 5 1|9 2 8|7 4 6|
+-----+-----+-----+
|1 2 8|5 3 7|6 9 4|
|6 3 4|8 9 2|1 5 7|
|7 9 5|4 6 1|8 3 2|
+-----+-----+-----+
|5 1 9|2 8 6|4 7 3|
|4 7 2|3 1 9|5 6 8|
|8 6 3|7 4 5|2 1 9|
+-----+-----+-----+
```

dlx(*count_only=False*)

Return a generator that iterates through all solutions of a Sudoku puzzle.

INPUT:

- `count_only` – boolean (default: `False`); if set to `True` the generator returned as output will simply generate `None` for each solution, so the calling routine can count these

OUTPUT: a generator that iterates over all the solutions

This function is intended to be called from the *`solve()`* method with the `algorithm='dlx'` option. However it may be called directly as a method of an instance of a Sudoku puzzle if speed is important and you do not need automatic conversions on the output (or even just want to count solutions without looking at them). In this case, inputting a puzzle as a list, with `verify_input=False` is the fastest way to create a puzzle.

Or if only one solution is needed it can be obtained with one call to `next()`, while the existence of a solution can be tested by catching the `StopIteration` exception with a `try`. Calling this particular method returns solutions as lists, in row-major order. It is up to you to work with this list for your own purposes. If you want fancier formatting tools, use the *solve()* method, which returns a generator that creates *sage.games.sudoku.Sudoku* objects.

EXAMPLES:

A 9×9 known to have one solution. We get the one solution and then check to see if there are more or not.

```
sage: e = Sudoku('4.....8.5.3..........7......2.....6.....8.4......1.......6.
↪3.7.5..2.....1.4......')
sage: print(next(e.dlx()))
[4, 1, 7, 3, 6, 9, 8, 2, 5, 6, 3, 2, 1, 5, 8, 9, 4, 7, 9, 5, 8, 7, 2, 4, 3, 1,
↪ 6, 8, 2, 5, 4, 3, 7, 1, 6, 9, 7, 9, 1, 5, 8, 6, 4, 3, 2, 3, 4, 6, 9, 1, 2,␣
↪7, 5, 8, 2, 8, 9, 6, 4, 3, 5, 7, 1, 5, 7, 3, 2, 9, 1, 6, 8, 4, 1, 6, 4, 8,␣
↪7, 5, 2, 9, 3]
sage: len(list(e.dlx()))
1
```

```
>>> from sage.all import *
>>> e = Sudoku('4.....8.5.3..........7......2.....6.....8.4......1.......6.3.
↪7.5..2.....1.4......')
>>> print(next(e.dlx()))
[4, 1, 7, 3, 6, 9, 8, 2, 5, 6, 3, 2, 1, 5, 8, 9, 4, 7, 9, 5, 8, 7, 2, 4, 3, 1,
↪ 6, 8, 2, 5, 4, 3, 7, 1, 6, 9, 7, 9, 1, 5, 8, 6, 4, 3, 2, 3, 4, 6, 9, 1, 2,␣
↪7, 5, 8, 2, 8, 9, 6, 4, 3, 5, 7, 1, 5, 7, 3, 2, 9, 1, 6, 8, 4, 1, 6, 4, 8,␣
↪7, 5, 2, 9, 3]
>>> len(list(e.dlx()))
1
```

A 9×9 puzzle with multiple solutions. Once with actual solutions, once just to count.

```
sage: h = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.
↪1..5...8..36..3......')
sage: len(list(h.dlx()))
5
sage: len(list(h.dlx(count_only=True)))
5
```

```
>>> from sage.all import *
>>> h = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.1.
↪.5...8..36..3......')
>>> len(list(h.dlx()))
5
>>> len(list(h.dlx(count_only=True)))
5
```

A larger puzzle, with multiple solutions, but we just get one.

```
sage: j = Sudoku('....a..69.3....1d.2...8....e.4....b....5..c.......7.......g.
↪..f....1.e..2.b.8..3.......4.d.....6.........f..7.g..9.a..c...5.....8..f....
↪.1..e.79.c....b.....2...6.....g.7......84....3.d..a.5....5...7..e...ca.....
↪3.1.......b......f....4...d..e..g.92.6..8....')
```

(continues on next page)

(continued from previous page)

```
sage: print(next(j.dlx()))
[5, 15, 16, 14, 10, 13, 7, 6, 9, 2, 3, 4, 11, 8, 12, 1, 13, 3, 2, 12, 11, 16,␣
↪8, 15, 1, 6, 7, 14, 10, 4, 9, 5, 1, 10, 11, 6, 9, 4, 3, 5, 15, 8, 12, 13,␣
↪16, 7, 14, 2, 9, 8, 7, 4, 12, 2, 1, 14, 10, 5, 16, 11, 6, 3, 15, 13, 12, 16,
↪ 4, 1, 13, 14, 9, 10, 2, 7, 11, 6, 8, 15, 5, 3, 3, 14, 5, 7, 16, 11, 15, 4,␣
↪12, 13, 8, 9, 1, 2, 10, 6, 2, 6, 13, 11, 1, 8, 5, 3, 4, 15, 14, 10, 7, 9,␣
↪16, 12, 15, 9, 8, 10, 2, 6, 12, 7, 3, 16, 5, 1, 4, 14, 13, 11, 8, 11, 3, 15,
↪ 5, 10, 4, 2, 13, 1, 6, 12, 14, 16, 7, 9, 16, 12, 14, 13, 7, 15, 11, 1, 8,␣
↪9, 4, 5, 2, 6, 3, 10, 6, 2, 10, 5, 14, 12, 16, 9, 7, 11, 15, 3, 13, 1, 4, 8,
↪ 4, 7, 1, 9, 8, 3, 6, 13, 16, 14, 10, 2, 5, 12, 11, 15, 11, 5, 9, 8, 6, 7,␣
↪13, 16, 14, 3, 1, 15, 12, 10, 2, 4, 7, 13, 15, 3, 4, 1, 10, 8, 5, 12, 2, 16,
↪ 9, 11, 6, 14, 10, 1, 6, 2, 15, 5, 14, 12, 11, 4, 9, 7, 3, 13, 8, 16, 14, 4,
↪ 12, 16, 3, 9, 2, 11, 6, 10, 13, 8, 15, 5, 1, 7]
```

```
>>> from sage.all import *
>>> j = Sudoku('....a..69.3....1d.2...8....e.4....b....5..c.......7.......g...
↪f....1.e..2.b.8..3.......4.d.....6.........f..7.g..9.a..c...5.....8..f.....
↪1..e.79.c....b.....2...6.....g.7......84....3.d..a.5....5...7..e...ca.....3.
↪1.......b......f....4...d..e..g.92.6..8....')
>>> print(next(j.dlx()))
[5, 15, 16, 14, 10, 13, 7, 6, 9, 2, 3, 4, 11, 8, 12, 1, 13, 3, 2, 12, 11, 16,␣
↪8, 15, 1, 6, 7, 14, 10, 4, 9, 5, 1, 10, 11, 6, 9, 4, 3, 5, 15, 8, 12, 13,␣
↪16, 7, 14, 2, 9, 8, 7, 4, 12, 2, 1, 14, 10, 5, 16, 11, 6, 3, 15, 13, 12, 16,
↪ 4, 1, 13, 14, 9, 10, 2, 7, 11, 6, 8, 15, 5, 3, 3, 14, 5, 7, 16, 11, 15, 4,␣
↪12, 13, 8, 9, 1, 2, 10, 6, 2, 6, 13, 11, 1, 8, 5, 3, 4, 15, 14, 10, 7, 9,␣
↪16, 12, 15, 9, 8, 10, 2, 6, 12, 7, 3, 16, 5, 1, 4, 14, 13, 11, 8, 11, 3, 15,
↪ 5, 10, 4, 2, 13, 1, 6, 12, 14, 16, 7, 9, 16, 12, 14, 13, 7, 15, 11, 1, 8,␣
↪9, 4, 5, 2, 6, 3, 10, 6, 2, 10, 5, 14, 12, 16, 9, 7, 11, 15, 3, 13, 1, 4, 8,
↪ 4, 7, 1, 9, 8, 3, 6, 13, 16, 14, 10, 2, 5, 12, 11, 15, 11, 5, 9, 8, 6, 7,␣
↪13, 16, 14, 3, 1, 15, 12, 10, 2, 4, 7, 13, 15, 3, 4, 1, 10, 8, 5, 12, 2, 16,
↪ 9, 11, 6, 14, 10, 1, 6, 2, 15, 5, 14, 12, 11, 4, 9, 7, 3, 13, 8, 16, 14, 4,
↪ 12, 16, 3, 9, 2, 11, 6, 10, 13, 8, 15, 5, 1, 7]
```

The puzzle `h` from above, but purposely made unsolvable with addition in second entry.

```
sage: hbad = Sudoku('82.6..9.5.............2.31...7318.6.24.....73..........
↪279.1..5...8..36..3......')
sage: len(list(hbad.dlx()))
0
sage: next(hbad.dlx())
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> hbad = Sudoku('82.6..9.5.............2.31...7318.6.24.....73..........
↪279.1..5...8..36..3......')
>>> len(list(hbad.dlx()))
0
>>> next(hbad.dlx())
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
StopIteration
```

A stupidly small puzzle to test the lower limits of arbitrary sized input.

```
sage: s = Sudoku('.')
sage: print(next(s.solve(algorithm='dlx')))
+-+
|1|
+-+
```

```
>>> from sage.all import *
>>> s = Sudoku('.')
>>> print(next(s.solve(algorithm='dlx')))
+-+
|1|
+-+
```

ALGORITHM:

The `DLXCPP` solver finds solutions to the exact-cover problem with a "Dancing Links" backtracking algorithm. Given a $0-1$ matrix, the solver finds a subset of the rows that sums to the all 1's vector. The columns correspond to conditions, or constraints, that must be met by a solution, while the rows correspond to some collection of choices, or decisions. A 1 in a row and column indicates that the choice corresponding to the row meets the condition corresponding to the column.

So here, we code the notion of a Sudoku puzzle, and the hints already present, into such a $0-1$ matrix. Then the `DLXCPP()` solver makes the choices for the blank entries.

solve(*algorithm='dlx'*)

Return a generator object for the solutions of a Sudoku puzzle.

INPUT:

- `algorithm` – string (default: `'dlx'`); specify choice of solution algorithm. The two possible algorithms are `'dlx'` and `'backtrack'`

OUTPUT:

A generator that provides all solutions, as objects of the *Sudoku* class.

Calling `next()` on the returned generator just once will find a solution, presuming it exists, otherwise it will return a `StopIteration` exception. The generator may be used for iteration or wrapping the generator with `list()` will return all of the solutions as a list. Solutions are returned as new objects of the *Sudoku* class, so may be printed or converted using other methods in this class.

Generally, the DLX algorithm is very fast and very consistent. The backtrack algorithm is very variable in its performance, on some occasions markedly faster than DLX but usually slower by a similar factor, with the potential to be orders of magnitude slower. See the docstrings for the *dlx()* and *backtrack()* methods for further discussions and examples of performance. Note that the backtrack algorithm is limited to puzzles of size 16×16 or smaller.

EXAMPLES:

This puzzle has 5 solutions, but the first one returned by each algorithm are identical.

```
sage: h = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.
↪1..5...8..36..3......')
```

(continues on next page)

(continued from previous page)

```
sage: h
+-----+-----+-----+
|8    |6    |9   5|
|     |     |     |
|     |  2  |3 1  |
+-----+-----+-----+
|    7|3 1 8|  6  |
|2 4  |     |  7 3|
|     |     |     |
+-----+-----+-----+
|    2|7 9  |1    |
|5    |  8  |  3 6|
|    3|     |     |
+-----+-----+-----+
sage: next(h.solve(algorithm='backtrack'))
+-----+-----+-----+
|8 1 4|6 3 7|9 2 5|
|3 2 5|1 4 9|6 8 7|
|7 9 6|8 2 5|3 1 4|
+-----+-----+-----+
|9 5 7|3 1 8|4 6 2|
|2 4 1|9 5 6|8 7 3|
|6 3 8|2 7 4|5 9 1|
+-----+-----+-----+
|4 6 2|7 9 3|1 5 8|
|5 7 9|4 8 1|2 3 6|
|1 8 3|5 6 2|7 4 9|
+-----+-----+-----+
sage: next(h.solve(algorithm='dlx'))
+-----+-----+-----+
|8 1 4|6 3 7|9 2 5|
|3 2 5|1 4 9|6 8 7|
|7 9 6|8 2 5|3 1 4|
+-----+-----+-----+
|9 5 7|3 1 8|4 6 2|
|2 4 1|9 5 6|8 7 3|
|6 3 8|2 7 4|5 9 1|
+-----+-----+-----+
|4 6 2|7 9 3|1 5 8|
|5 7 9|4 8 1|2 3 6|
|1 8 3|5 6 2|7 4 9|
+-----+-----+-----+
```

```
>>> from sage.all import *
>>> h = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.1.
↪.5...8..36..3......')
>>> h
+-----+-----+-----+
|8    |6    |9   5|
|     |     |     |
|     |  2  |3 1  |
+-----+-----+-----+
```

(continues on next page)

(continued from previous page)

```
|    7|3 1 8|  6  |
|2 4  |     |  7 3|
|     |     |     |
+-----+-----+-----+
|    2|7 9  |1    |
|5    |  8  |  3 6|
|    3|     |     |
+-----+-----+-----+
>>> next(h.solve(algorithm='backtrack'))
+-----+-----+-----+
|8 1 4|6 3 7|9 2 5|
|3 2 5|1 4 9|6 8 7|
|7 9 6|8 2 5|3 1 4|
+-----+-----+-----+
|9 5 7|3 1 8|4 6 2|
|2 4 1|9 5 6|8 7 3|
|6 3 8|2 7 4|5 9 1|
+-----+-----+-----+
|4 6 2|7 9 3|1 5 8|
|5 7 9|4 8 1|2 3 6|
|1 8 3|5 6 2|7 4 9|
+-----+-----+-----+
>>> next(h.solve(algorithm='dlx'))
+-----+-----+-----+
|8 1 4|6 3 7|9 2 5|
|3 2 5|1 4 9|6 8 7|
|7 9 6|8 2 5|3 1 4|
+-----+-----+-----+
|9 5 7|3 1 8|4 6 2|
|2 4 1|9 5 6|8 7 3|
|6 3 8|2 7 4|5 9 1|
+-----+-----+-----+
|4 6 2|7 9 3|1 5 8|
|5 7 9|4 8 1|2 3 6|
|1 8 3|5 6 2|7 4 9|
+-----+-----+-----+
```

Gordon Royle maintains a list of 48072 Sudoku puzzles that each has a unique solution and exactly 17 "hints" (initially filled boxes). At this writing (May 2009) there is no known 16-hint puzzle with exactly one solution. [sudoku:royle] This puzzle is number 3000 in his database. We solve it twice.

```
sage: b = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.
↪1..5...8..36..3......')
sage: next(b.solve(algorithm='dlx')) == next(b.solve(algorithm='backtrack'))
True
```

```
>>> from sage.all import *
>>> b = Sudoku('8..6..9.5.............2.31...7318.6.24.....73..........279.1.
↪.5...8..36..3......')
>>> next(b.solve(algorithm='dlx')) == next(b.solve(algorithm='backtrack'))
True
```

These are the first 10 puzzles in a list of "Top 95" puzzles, [sudoku:top95] which we use to show that the two

available algorithms obtain the same solution for each.

```
sage: top =['4.....8.5.3.........7......2.....6.....8.4......1.......6.3.7.5.
↪.2.....1.4......',
....:       '52...6.........7.13...........4..8..6......5...........418.......
↪..3..2...87.....',
....:       '6.....8.3.4.7.................5.4.7.3..2.....1.6.......2.....5...
↪..8.6......1....',
....:       '48.3............71.2.......7.5....6....2..8............1.76...3.
↪....4......5....',
....:       '....14....3....2...7..........9...3.6.1.............8.2.....1.4..
↪..5.6.....7.8...',
....:       '......52..8.4......3...9...5.1...6..2..7........3.....6...1.....
↪....7.4.......3.',
....:       '6.2.5.........3.4..........43...8....1....2........7..5..27......
↪.....81...6.....',
....:       '.524.........7.1..............8.2...3.....6...9.5.....1.6.3......
↪.....897........',
....:       '6.2.5.........4.3..........43...8....1....2........7..5..27......
↪.....81...6.....',
....:       '.923.........8.1...........1.7.4...........658.........6.5.2...4.
↪....7.....9.....']
sage: p = [Sudoku(top[i]) for i in range(10)]
sage: verify = [next(p[i].solve(algorithm='dlx')) == next(p[i].
↪solve(algorithm='backtrack')) for i in range(10)]
sage: verify == [True]*10
True
```

```
>>> from sage.all import *
>>> top =['4.....8.5.3.........7......2.....6.....8.4......1.......6.3.7.5..
↪2.....1.4......',
...       '52...6.........7.13...........4..8..6......5...........418.........
↪3..2...87.....',
...       '6.....8.3.4.7.................5.4.7.3..2.....1.6.......2.....5.....
↪8.6......1....',
...       '48.3............71.2.......7.5....6....2..8............1.76...3...
↪..4......5....',
...       '....14....3....2...7..........9...3.6.1.............8.2.....1.4....
↪5.6.....7.8...',
...       '......52..8.4......3...9...5.1...6..2..7........3.....6...1........
↪..7.4.......3.',
...       '6.2.5.........3.4..........43...8....1....2........7..5..27........
↪...81...6.....',
...       '.524.........7.1..............8.2...3.....6...9.5.....1.6.3........
↪...897........',
...       '6.2.5.........4.3..........43...8....1....2........7..5..27........
↪...81...6.....',
...       '.923.........8.1...........1.7.4...........658.........6.5.2...4...
↪..7.....9.....']
>>> p = [Sudoku(top[i]) for i in range(Integer(10))]
>>> verify = [next(p[i].solve(algorithm='dlx')) == next(p[i].solve(algorithm=
↪'backtrack')) for i in range(Integer(10))]
>>> verify == [True]*Integer(10)
```

(continues on next page)

(continued from previous page)

```
True
```

to_ascii()

Construct an ASCII-art version of a Sudoku puzzle. This is a modified version of the ASCII version of a subdivided matrix.

EXAMPLES:

```
sage: s = Sudoku('.4..32....14..3.')
sage: print(s.to_ascii())
+---+---+
|  4|   |
|3 2|   |
+---+---+
|   |1 4|
|   |3  |
+---+---+
sage: s.to_ascii()
'+---+---+\n|  4|   |\n|3 2|   |\n+---+---+\n|   |1 4|\n|   |3  |\n+---+---+'
```

```
>>> from sage.all import *
>>> s = Sudoku('.4..32....14..3.')
>>> print(s.to_ascii())
+---+---+
|  4|   |
|3 2|   |
+---+---+
|   |1 4|
|   |3  |
+---+---+
>>> s.to_ascii()
'+---+---+\n|  4|   |\n|3 2|   |\n+---+---+\n|   |1 4|\n|   |3  |\n+---+---+'
```

to_latex()

Create a string of LaTeX code representing a Sudoku puzzle or solution.

EXAMPLES:

```
sage: s = Sudoku('.4..32....14..3.')
sage: print(s.to_latex())
\begin{array}{|*{2}{*{2}{r|}}\hline
&4& & \\
3&2& & \\\hline
& &1&4\\
& &3& \\\hline
\end{array}
```

```
>>> from sage.all import *
>>> s = Sudoku('.4..32....14..3.')
>>> print(s.to_latex())
\begin{array}{|*{2}{*{2}{r|}}\hline
&4& & \\
```

(continues on next page)

(continued from previous page)

```
3&2& & \\\hline
& &1&4\\
& &3& \\\hline
\end{array}
```

to_list()

Construct a list representing a Sudoku puzzle, in row-major order.

EXAMPLES:

```
sage: s = Sudoku('1.......2.9.4...5...6...7...5.9.3.......7.......85..4.7.....
↪6...3...9.8...2.....1')
sage: s.to_list()
[1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 9, 0, 4, 0, 0, 0, 5, 0, 0, 0, 6, 0, 0, 0, 7, 0,
↪ 0, 0, 5, 0, 9, 0, 3, 0, 0, 0, 0, 0, 0, 0, 7, 0, 0, 0, 0, 0, 0, 0, 8, 5, 0,␣
↪0, 4, 0, 7, 0, 0, 0, 0, 0, 6, 0, 0, 0, 3, 0, 0, 0, 9, 0, 8, 0, 0, 0, 2, 0,␣
↪0, 0, 0, 0, 1]
```

```
>>> from sage.all import *
>>> s = Sudoku('1.......2.9.4...5...6...7...5.9.3.......7.......85..4.7.....6.
↪..3...9.8...2.....1')
>>> s.to_list()
[1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 9, 0, 4, 0, 0, 0, 5, 0, 0, 0, 6, 0, 0, 0, 7, 0,
↪ 0, 0, 5, 0, 9, 0, 3, 0, 0, 0, 0, 0, 0, 0, 7, 0, 0, 0, 0, 0, 0, 0, 8, 5, 0,␣
↪0, 4, 0, 7, 0, 0, 0, 0, 0, 6, 0, 0, 0, 3, 0, 0, 0, 9, 0, 8, 0, 0, 0, 2, 0,␣
↪0, 0, 0, 0, 1]
```

to_matrix()

Construct a Sage matrix over **Z** representing a Sudoku puzzle.

EXAMPLES:

```
sage: s = Sudoku('.4..32....14..3.')
sage: s.to_matrix()
[0 4 0 0]
[3 2 0 0]
[0 0 1 4]
[0 0 3 0]
```

```
>>> from sage.all import *
>>> s = Sudoku('.4..32....14..3.')
>>> s.to_matrix()
[0 4 0 0]
[3 2 0 0]
[0 0 1 4]
[0 0 3 0]
```

to_string()

Construct a string representing a Sudoku puzzle.

Blank entries are represented as periods, single digits are not converted and two digit entries are converted to lower-case letters where `10 = a`, `11 = b`, etc. This scheme limits puzzles to at most 36 symbols.

EXAMPLES:

```
sage: b = matrix(ZZ, 9, 9, [ [0,0,0,0,1,0,9,0,0], [8,0,0,4,0,0,0,0,0], [2,0,0,
↪0,0,0,0,0,0], [0,7,0,0,3,0,0,0,0], [0,0,0,0,0,0,2,0,4], [0,0,0,0,0,0,0,5,8],
↪ [0,6,0,0,0,0,1,3,0], [7,0,0,2,0,0,0,0,0], [0,0,0,8,0,0,0,0,0] ])
sage: Sudoku(b).to_string()
'....1.9..8..4.....2.........7..3..........2.4.......58.6....13.7..2........8.
↪....'
```

```
>>> from sage.all import *
>>> b = matrix(ZZ, Integer(9), Integer(9), [ [Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(1),Integer(0),Integer(9),Integer(0),
↪Integer(0)], [Integer(8),Integer(0),Integer(0),Integer(4),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0)], [Integer(2),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(0)], [Integer(0),Integer(7),Integer(0),Integer(0),Integer(3),
↪Integer(0),Integer(0),Integer(0),Integer(0)], [Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(2),Integer(0),
↪Integer(4)], [Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(5),Integer(8)], [Integer(0),Integer(6),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(1),Integer(3),
↪Integer(0)], [Integer(7),Integer(0),Integer(0),Integer(2),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0)], [Integer(0),Integer(0),
↪Integer(0),Integer(8),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(0)] ])
>>> Sudoku(b).to_string()
'....1.9..8..4.....2.........7..3..........2.4.......58.6....13.7..2........8.
↪....'
```

sage.games.sudoku.**sudoku**(*m*)

Solve Sudoku puzzles described by matrices.

INPUT:

- `m` – a square Sage matrix over **Z**, where zeros are blank entries

OUTPUT:

A Sage matrix over **Z** containing the first solution found, otherwise `None`.

This function matches the behavior of the prior Sudoku solver and is included only to replicate that behavior. It could be safely deprecated, since all of its functionality is included in the *Sudoku* class.

EXAMPLES:

An example that was used in previous doctests.

```
sage: A = matrix(ZZ,9,[5,0,0, 0,8,0, 0,4,9, 0,0,0, 5,0,0, 0,3,0, 0,6,7, 3,0,0, 0,
↪0,1, 1,5,0, 0,0,0, 0,0,0,  0,0,0, 2,0,8, 0,0,0, 0,0,0, 0,0,0, 0,1,8, 7,0,0, 0,0,
↪4, 1,5,0, 0,3,0, 0,0,2, 0,0,0, 4,9,0, 0,5,0, 0,0,3])
sage: A
[5 0 0 0 8 0 0 4 9]
[0 0 0 5 0 0 0 3 0]
[0 6 7 3 0 0 0 0 1]
[1 5 0 0 0 0 0 0 0]
[0 0 0 2 0 8 0 0 0]
[0 0 0 0 0 0 0 1 8]
[7 0 0 0 0 4 1 5 0]
```

(continues on next page)

(continued from previous page)

```
[0 3 0 0 0 2 0 0 0]
[4 9 0 0 5 0 0 0 3]
sage: sudoku(A)
[5 1 3 6 8 7 2 4 9]
[8 4 9 5 2 1 6 3 7]
[2 6 7 3 4 9 5 8 1]
[1 5 8 4 6 3 9 7 2]
[9 7 4 2 1 8 3 6 5]
[3 2 6 7 9 5 4 1 8]
[7 8 2 9 3 4 1 5 6]
[6 3 5 1 7 2 8 9 4]
[4 9 1 8 5 6 7 2 3]
```

```
>>> from sage.all import *
>>> A = matrix(ZZ,Integer(9),[Integer(5),Integer(0),Integer(0), Integer(0),
↪Integer(8),Integer(0), Integer(0),Integer(4),Integer(9), Integer(0),Integer(0),
↪Integer(0), Integer(5),Integer(0),Integer(0), Integer(0),Integer(3),Integer(0),␣
↪Integer(0),Integer(6),Integer(7), Integer(3),Integer(0),Integer(0), Integer(0),
↪Integer(0),Integer(1), Integer(1),Integer(5),Integer(0), Integer(0),Integer(0),
↪Integer(0), Integer(0),Integer(0),Integer(0),  Integer(0),Integer(0),Integer(0),
↪ Integer(2),Integer(0),Integer(8), Integer(0),Integer(0),Integer(0), Integer(0),
↪Integer(0),Integer(0), Integer(0),Integer(0),Integer(0), Integer(0),Integer(1),
↪Integer(8), Integer(7),Integer(0),Integer(0), Integer(0),Integer(0),Integer(4),␣
↪Integer(1),Integer(5),Integer(0), Integer(0),Integer(3),Integer(0), Integer(0),
↪Integer(0),Integer(2), Integer(0),Integer(0),Integer(0), Integer(4),Integer(9),
↪Integer(0), Integer(0),Integer(5),Integer(0), Integer(0),Integer(0),Integer(3)])
>>> A
[5 0 0 0 8 0 0 4 9]
[0 0 0 5 0 0 0 3 0]
[0 6 7 3 0 0 0 0 1]
[1 5 0 0 0 0 0 0 0]
[0 0 0 2 0 8 0 0 0]
[0 0 0 0 0 0 0 1 8]
[7 0 0 0 0 4 1 5 0]
[0 3 0 0 0 2 0 0 0]
[4 9 0 0 5 0 0 0 3]
>>> sudoku(A)
[5 1 3 6 8 7 2 4 9]
[8 4 9 5 2 1 6 3 7]
[2 6 7 3 4 9 5 8 1]
[1 5 8 4 6 3 9 7 2]
[9 7 4 2 1 8 3 6 5]
[3 2 6 7 9 5 4 1 8]
[7 8 2 9 3 4 1 5 6]
[6 3 5 1 7 2 8 9 4]
[4 9 1 8 5 6 7 2 3]
```

Using inputs that are possible with the *Sudoku* class, other than a matrix, will cause an error.

```
sage: sudoku('.4..32....14..3.')
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: sudoku function expects puzzle to be a matrix, perhaps use the Sudoku␣
↪class
```

```
>>> from sage.all import *
>>> sudoku('.4..32....14..3.')
Traceback (most recent call last):
...
ValueError: sudoku function expects puzzle to be a matrix, perhaps use the Sudoku␣
↪class
```

CHAPTER

TWO

FAMILY GAMES AMERICA'S QUANTUMINO SOLVER

This module allows to solve the Quantumino puzzle made by Family Games America (see also this video on Youtube). This puzzle was left at the dinner room of the Laboratoire de Combinatoire Informatique Mathématique in Montreal by Franco Saliola during winter 2011.

The solution uses the dancing links code which is in Sage and is based on the more general code available in the module `sage.combinat.tiling`. Dancing links were originally introduced by Donald Knuth in 2000 (arXiv cs/0011047). In particular, Knuth used dancing links to solve tilings of a region by 2D pentaminos. Here we extend the method for 3D pentaminos.

This module defines two classes :

- *sage.games.quantumino.QuantuminoState* class, to represent a state of the Quantumino game, i.e. a solution or a partial solution.
- *sage.games.quantumino.QuantuminoSolver* class, to find, enumerate and count the number of solutions of the Quantumino game where one of the piece is put aside.

AUTHOR:

- Sébastien Labbé, April 28th, 2011

DESCRIPTION (from [1]):

> " Pentamino games have been taken to a whole different level; a 3-D level, with this colorful creation! Using the original pentamino arrangements of 5 connected squares which date from 1907, players are encouraged to "think inside the box" as they try to fit 16 of the 17 3-D pentamino pieces inside the playing perimeters. Remove a different piece each time you play for an entirely new challenge! Thousands of solutions to be found! Quantumino hands-on educational tool where players learn how shapes can be transformed or arranged into predefined shapes and spaces. Includes: 1 wooden frame, 17 wooden blocks, instruction booklet. Age: 8+ "

EXAMPLES:

Here are the 17 wooden blocks of the Quantumino puzzle numbered from 0 to 16 in the following 3d picture. They will show up in 3D in your default (=Jmol) viewer:

```
sage: from sage.games.quantumino import show_pentaminos
sage: show_pentaminos()
Graphics3d Object
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import show_pentaminos
>>> show_pentaminos()
Graphics3d Object
```

To solve the puzzle where the pentamino numbered 12 is put aside:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: s = next(QuantuminoSolver(12).solve())          # long time (10 s)
sage: s                                               # long time (<1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (2, 1, 1)], Color: blue
sage: s.show3d()                                      # long time (<1s)
Graphics3d Object
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> s = next(QuantuminoSolver(Integer(12)).solve())          # long time (10 s)
>>> s                                                 # long time (<1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (2, 1, 1)], Color: blue
>>> s.show3d()                                        # long time (<1s)
Graphics3d Object
```

To remove the frame:

```
sage: s.show3d().show(frame=False)                    # long time (<1s)
```

```
>>> from sage.all import *
>>> s.show3d().show(frame=False)                      # long time (<1s)
```

To solve the puzzle where the pentamino numbered 7 is put aside:

```
sage: s = next(QuantuminoSolver(7).solve())           # long time (10 s)
sage: s                                               # long time (<1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
sage: s.show3d()                                      # long time (<1s)
Graphics3d Object
```

```
>>> from sage.all import *
>>> s = next(QuantuminoSolver(Integer(7)).solve())           # long time (10 s)
>>> s                                                 # long time (<1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
>>> s.show3d()                                        # long time (<1s)
Graphics3d Object
```

The solution is iterable. This may be used to explicitly list the positions of each pentamino:

```
sage: for p in s: p                                   # long time (<1s)
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
```

(continues on next page)

(continued from previous page)

```
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
```

```
>>> from sage.all import *
>>> for p in s: p                                    # long time (<1s)
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
```

To get all the solutions, use the iterator returned by the `solve` method. Note that finding the first solution is the most time consuming because it needs to create the complete data to describe the problem:

```
sage: it = QuantuminoSolver(7).solve()
sage: next(it)                                    # not tested (10s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
sage: next(it)                                    # not tested (0.001s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
sage: next(it)                                    # not tested (0.001s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
```

```
>>> from sage.all import *
>>> it = QuantuminoSolver(Integer(7)).solve()
>>> next(it)                                      # not tested (10s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
>>> next(it)                                      # not tested (0.001s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
>>> next(it)                                      # not tested (0.001s)
Quantumino state where the following pentamino is put aside :
```

(continues on next page)

(continued from previous page)

```
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color: orange
```

To get the solution inside other boxes:

```
sage: s = next(QuantuminoSolver(7, box=(4,4,5)).solve())          # not tested (2s)
sage: s.show3d()                                                  # not tested (<1s)
```

```
>>> from sage.all import *
>>> s = next(QuantuminoSolver(Integer(7), box=(Integer(4),Integer(4),Integer(5))).
↪solve())        # not tested (2s)
>>> s.show3d()                                                  # not tested (<1s)
```

```
sage: s = next(QuantuminoSolver(7, box=(2,2,20)).solve())         # not tested (1s)
sage: s.show3d()                                                  # not tested (<1s)
```

```
>>> from sage.all import *
>>> s = next(QuantuminoSolver(Integer(7), box=(Integer(2),Integer(2),Integer(20))).
↪solve())       # not tested (1s)
>>> s.show3d()                                                  # not tested (<1s)
```

If there are no solution, a StopIteration error is raised:

```
sage: next(QuantuminoSolver(7, box=(3,3,3)).solve())
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> next(QuantuminoSolver(Integer(7), box=(Integer(3),Integer(3),Integer(3))).solve())
Traceback (most recent call last):
...
StopIteration
```

The implementation allows a lot of introspection. From the `TilingSolver` object, it is possible to retrieve the rows that are passed to the DLX solver and count them. It is also possible to get an instance of the DLX solver to play with it:

```
sage: q = QuantuminoSolver(0)
sage: T = q.tiling_solver()
sage: T
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
sage: rows = T.rows()                                 # not tested (10 s)
sage: len(rows)                                       # not tested (but fast)
5484
sage: x = T.dlx_solver()                              # long time (10 s)
sage: x                                               # long time (fast)
Dancing links solver for 96 columns and 5484 rows
```

```
>>> from sage.all import *
>>> q = QuantuminoSolver(Integer(0))
>>> T = q.tiling_solver()
>>> T
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
>>> rows = T.rows()                              # not tested (10 s)
>>> len(rows)                                    # not tested (but fast)
5484
>>> x = T.dlx_solver()                           # long time (10 s)
>>> x                                            # long time (fast)
Dancing links solver for 96 columns and 5484 rows
```

REFERENCES:

- [1] Family Games America's Quantumino
- [2] Quantumino - How to Play on Youtube
- [3] Knuth, Donald (2000). *Dancing links*. arXiv cs/0011047.

class sage.games.quantumino.**QuantuminoSolver**(*aside*, *box=(5, 8, 2)*)

Bases: SageObject

Return the Quantumino solver for the given box where one of the pentamino is put aside.

INPUT:

- `aside` – integer, from 0 to 16, the aside pentamino
- `box` – tuple of size three (default: `(5,8,2)`), size of the box

EXAMPLES:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: QuantuminoSolver(9)
Quantumino solver for the box (5, 8, 2)
Aside pentamino number: 9
sage: QuantuminoSolver(12, box=(5,4,4))
Quantumino solver for the box (5, 4, 4)
Aside pentamino number: 12
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> QuantuminoSolver(Integer(9))
Quantumino solver for the box (5, 8, 2)
Aside pentamino number: 9
>>> QuantuminoSolver(Integer(12), box=(Integer(5),Integer(4),Integer(4)))
Quantumino solver for the box (5, 4, 4)
Aside pentamino number: 12
```

number_of_solutions()

Return the number of solutions.

OUTPUT: integer

EXAMPLES:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: QuantuminoSolver(4, box=(3,2,2)).number_of_solutions()
0
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> QuantuminoSolver(Integer(4), box=(Integer(3),Integer(2),Integer(2))).
↪number_of_solutions()
0
```

This computation takes several days:

```
sage: QuantuminoSolver(0).number_of_solutions()                # not tested
??? hundreds of millions ???
```

```
>>> from sage.all import *
>>> QuantuminoSolver(Integer(0)).number_of_solutions()                # not␣
↪tested
??? hundreds of millions ???
```

solve(*partial=None*)

Return an iterator over the solutions where one of the pentamino is put aside.

INPUT:

- `partial` – string (default: `None`), whether to include partial (incomplete) solutions. It can be one of the following:
  - `None` – include only complete solution
  - `'common'` – common part between two consecutive solutions
  - `'incremental'` – one piece change at a time

OUTPUT: iterator of *QuantuminoState*

EXAMPLES:

Get one solution:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: s = next(QuantuminoSolver(8).solve())          # long time (9s)
sage: s                                              # long time (fast)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 1, 0)], Color:␣
↪yellow
sage: s.show3d()                                     # long time (< 1s)
Graphics3d Object
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> s = next(QuantuminoSolver(Integer(8)).solve())          # long time (9s)
>>> s                                                  # long time (fast)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 1, 0)], Color:␣
```

(continues on next page)

(continued from previous page)

```
↪yellow
>>> s.show3d()                                    # long time (< 1s)
Graphics3d Object
```

The explicit solution:

```
sage: for p in s: p                                  # long time (fast)
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
```

```
>>> from sage.all import *
>>> for p in s: p                                    # long time (fast)
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
Polyomino: [(...), (...), (...), (...), (...)], Color: ...
```

Enumerate the solutions:

```
sage: it = QuantuminoSolver(0).solve()
sage: next(it)                                       # not tested
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
sage: next(it)                                       # not tested
Quantumino state where the following pentamino is put aside :
```

(continues on next page)

(continued from previous page)

```
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
```

```
>>> from sage.all import *
>>> it = QuantuminoSolver(Integer(0)).solve()
>>> next(it)                                              # not tested
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
>>> next(it)                                              # not tested
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
```

With the partial solutions included, one can see the evolution between consecutive solutions (an animation would be better):

```
sage: it = QuantuminoSolver(0).solve(partial='common')
sage: next(it).show3d()                # not tested (2s)
sage: next(it).show3d()                # not tested (< 1s)
sage: next(it).show3d()                # not tested (< 1s)
```

```
>>> from sage.all import *
>>> it = QuantuminoSolver(Integer(0)).solve(partial='common')
>>> next(it).show3d()                # not tested (2s)
>>> next(it).show3d()                # not tested (< 1s)
>>> next(it).show3d()                # not tested (< 1s)
```

Generalizations of the game inside different boxes:

```
sage: next(QuantuminoSolver(7, (4,4,5)).solve())       # long time (2s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color:␣
↪orange
sage: next(QuantuminoSolver(7, (2,2,20)).solve())      # long time (1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color:␣
↪orange
sage: next(QuantuminoSolver(3, (2,2,20)).solve())      # long time (1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (1, 0, 0), (1, 0, 1)], Color:␣
↪green
```

```
>>> from sage.all import *
>>> next(QuantuminoSolver(Integer(7), (Integer(4),Integer(4),Integer(5))).
↪solve())       # long time (2s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color:␣
↪orange
>>> next(QuantuminoSolver(Integer(7), (Integer(2),Integer(2),Integer(20))).
↪solve())      # long time (1s)
Quantumino state where the following pentamino is put aside :
```

(continues on next page)

(continued from previous page)

```
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (0, 2, 1), (1, 0, 0)], Color:␣
↪orange
>>> next(QuantuminoSolver(Integer(3), (Integer(2),Integer(2),Integer(20))).
↪solve())      # long time (1s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (1, 0, 0), (1, 0, 1)], Color:␣
↪green
```

If the volume of the box is not 80, there is no solution:

```
sage: next(QuantuminoSolver(7, box=(3,3,9)).solve())
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> next(QuantuminoSolver(Integer(7), box=(Integer(3),Integer(3),Integer(9))).
↪solve())
Traceback (most recent call last):
...
StopIteration
```

If the box is too small, there is no solution:

```
sage: next(QuantuminoSolver(4, box=(40,2,1)).solve())
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> next(QuantuminoSolver(Integer(4), box=(Integer(40),Integer(2),
↪Integer(1))).solve())
Traceback (most recent call last):
...
StopIteration
```

tiling_solver()

Return the Tiling solver of the Quantumino Game where one of the pentamino is put aside.

EXAMPLES:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: QuantuminoSolver(0).tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
sage: QuantuminoSolver(14).tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
```

(continues on next page)

(continued from previous page)

```
sage: QuantuminoSolver(14, box=(5,4,4)).tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> QuantuminoSolver(Integer(0)).tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
>>> QuantuminoSolver(Integer(14)).tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
>>> QuantuminoSolver(Integer(14), box=(Integer(5),Integer(4),Integer(4))).
↪tiling_solver()
Tiling solver of 16 pieces into a box of size 80
Rotation allowed: True
Reflection allowed: False
Reusing pieces allowed: False
```

class sage.games.quantumino.**QuantuminoState**(*pentos*, *aside*, *box=(5, 8, 2)*)

Bases: SageObject

A state of the Quantumino puzzle.

Used to represent a solution or a partial solution of the Quantumino puzzle.

INPUT:

- `pentos` – list of 16 3d pentamino representing the (partial) solution
- `aside` – 3d polyomino, the unused 3D pentamino
- `box` – tuple of size three (default: `(5,8,2)`), size of the box

EXAMPLES:

```
sage: from sage.games.quantumino import pentaminos, QuantuminoState
sage: p = pentaminos[0]
sage: q = pentaminos[5]
sage: r = pentaminos[11]
sage: S = QuantuminoState([p,q], r)
sage: S
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 2, 0)], Color:␣
↪darkblue
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import pentaminos, QuantuminoState
>>> p = pentaminos[Integer(0)]
```

(continues on next page)

(continued from previous page)

```
>>> q = pentaminos[Integer(5)]
>>> r = pentaminos[Integer(11)]
>>> S = QuantuminoState([p,q], r)
>>> S
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 2, 0)], Color:␣
↪darkblue
```

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: next(QuantuminoSolver(3).solve())       # not tested (1.5s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (1, 0, 0), (1, 0, 1)], Color: green
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> next(QuantuminoSolver(Integer(3)).solve())       # not tested (1.5s)
Quantumino state where the following pentamino is put aside :
Polyomino: [(0, 0, 0), (0, 1, 0), (0, 2, 0), (1, 0, 0), (1, 0, 1)], Color: green
```

list()

Return the list of 3d polyomino making the solution.

EXAMPLES:

```
sage: from sage.games.quantumino import pentaminos, QuantuminoState
sage: p = pentaminos[0]
sage: q = pentaminos[5]
sage: r = pentaminos[11]
sage: S = QuantuminoState([p,q], r)
sage: L = S.list()
sage: L[0]
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
sage: L[1]
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 0, 1), (1, 1, 0), (2, 0, 1)], Color: red
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import pentaminos, QuantuminoState
>>> p = pentaminos[Integer(0)]
>>> q = pentaminos[Integer(5)]
>>> r = pentaminos[Integer(11)]
>>> S = QuantuminoState([p,q], r)
>>> L = S.list()
>>> L[Integer(0)]
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 1, 0), (1, 1, 1), (1, 2, 0)], Color:␣
↪deeppink
>>> L[Integer(1)]
Polyomino: [(0, 0, 0), (1, 0, 0), (1, 0, 1), (1, 1, 0), (2, 0, 1)], Color: red
```

show3d(*size=0.85*)

Return the solution as a 3D Graphic object.

OUTPUT: 3D Graphic Object

EXAMPLES:

```
sage: from sage.games.quantumino import QuantuminoSolver
sage: s = next(QuantuminoSolver(0).solve())     # not tested (1.5s)
sage: G = s.show3d()                             # not tested (<1s)
sage: type(G)                                    # not tested
<class 'sage.plot.plot3d.base.Graphics3dGroup'>
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import QuantuminoSolver
>>> s = next(QuantuminoSolver(Integer(0)).solve())     # not tested (1.5s)
>>> G = s.show3d()                                    # not tested (<1s)
>>> type(G)                                           # not tested
<class 'sage.plot.plot3d.base.Graphics3dGroup'>
```

To remove the frame:

```
sage: G.show(frame=False) # not tested
```

```
>>> from sage.all import *
>>> G.show(frame=False) # not tested
```

To see the solution with Tachyon viewer:

```
sage: G.show(viewer='tachyon', frame=False) # not tested
```

```
>>> from sage.all import *
>>> G.show(viewer='tachyon', frame=False) # not tested
```

sage.games.quantumino.**show_pentaminos**(*box=(5, 8, 2)*)

Show the 17 3-D pentaminos included in the game and the $5 \times 8 \times 2$ box where 16 of them must fit.

INPUT:

- `box` – tuple of size three (default: `(5,8,2)`), size of the box

OUTPUT: 3D Graphic object

EXAMPLES:

```
sage: from sage.games.quantumino import show_pentaminos
sage: show_pentaminos()     # not tested (1s)
```

```
>>> from sage.all import *
>>> from sage.games.quantumino import show_pentaminos
>>> show_pentaminos()     # not tested (1s)
```

To remove the frame do:

```
sage: show_pentaminos().show(frame=False)  # not tested (1s)
```

```
>>> from sage.all import *
>>> show_pentaminos().show(frame=False)  # not tested (1s)
```

CHAPTER

THREE

HEXADS IN S(5, 6, 12)

This module completes a 5-element subset of a 12-set X into a hexad in a Steiner system $S(5, 6, 12)$ using Curtis and Conway's "kitten method". The labeling is either the "modulo 11" labeling or the "shuffle" labeling.

The main functions implemented in this file are *Minimog.blackjack_move()* and *Minimog.find_hexad()*.

Enter `blackjack_move?` for help to play blackjack (i.e., the rules of the game), or `find_hexad?` for help finding hexads of $S(5, 6, 12)$ in the shuffle labeling.

This picture is the kitten in the "shuffle" labeling:

```
                    6

                    9

                10      8

            7       2       5

        9       4       11      9

    10      8       3       10      8

1                                       0
```

The corresponding MINIMOG is:

```
┌─────┬─────┬─────┬─────┐
│  6  │  3  │  0  │  9  │
├─────┼─────┼─────┼─────┤
│  5  │  2  │  7  │ 10  │
├─────┼─────┼─────┼─────┤
│  4  │  1  │  8  │ 11  │
└─────┴─────┴─────┴─────┘
```

which is specified by the global variable `minimog_shuffle`.

See the docstrings for *Minimog.find_hexad()* and *Minimog.blackjack_move()* for further details and examples.

AUTHOR:

David Joyner (2006-05)

REFERENCES:

- [Cu1984]

- [Co1984]
- [CS1986]
- [KR2001]

Some details are also online at: https://www.permutationpuzzles.org/hexad/

class sage.games.hexad.**Minimog**(*type='shuffle'*)

Bases: object

This implements the Conway/Curtis minimog idea for describing the Steiner triple system $S(5, 6, 12)$.

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: Minimog(type='shuffle')
Minimog of type shuffle
sage: M = Minimog(type = "modulo11")
sage: M.minimog
[         0          3 +Infinity          2]
[         5          9          8         10]
[         4          1          6          7]
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> Minimog(type='shuffle')
Minimog of type shuffle
>>> M = Minimog(type = "modulo11")
>>> M.minimog
[         0          3 +Infinity          2]
[         5          9          8         10]
[         4          1          6          7]
```

blackjack_move(*L0*)

Perform a blackjack move.

INPUT:

- L0 – list of cards of length 6, taken from $\{0, 1, ..., 11\}$

MATHEMATICAL BLACKJACK

Mathematical blackjack is played with 12 cards, labeled $0, ..., 11$ (for example: king, ace, 2, 3, …, 10, jack, where the king is 0 and the jack is 11). Divide the 12 cards into two piles of 6 (to be fair, this should be done randomly). Each of the 6 cards of one of these piles are to be placed face up on the table. The remaining cards are in a stack which is shared and visible to both players. If the sum of the cards face up on the table is less than 21 then no legal move is possible so you must shuffle the cards and deal a new game. (Conway calls such a game $* = 0|0$, where $0 = |$; in this game the first player automatically wins.)

- Players alternate moves.
- A move consists of exchanging a card on the table with a lower card from the other pile.
- The player whose move makes the sum of the cards on the table under 21 loses.

The winning strategy (given below) for this game is due to Conway and Ryba. There is a Steiner system $S(5, 6, 12)$ of hexads in the set $\{0, 1, ..., 11\}$. This Steiner system is associated to the MINIMOG of in the "shuffle numbering" rather than the "modulo 11 labeling".

Proposition ([KR2001]) For this Steiner system, the winning strategy is to choose a move which is a hexad from this system.

EXAMPLES:

```
sage: M = Minimog(type='modulo11')
sage: M.blackjack_move([0, 2, 3, 6, 1, 10])
'6 --> 5. The total went from 22 to 21.'
sage: M = Minimog(type='shuffle')
sage: M.blackjack_move([0, 2, 4, 6, 7, 11])
'4 --> 3. The total went from 30 to 29.'
```

```
>>> from sage.all import *
>>> M = Minimog(type='modulo11')
>>> M.blackjack_move([Integer(0), Integer(2), Integer(3), Integer(6),␣
↪Integer(1), Integer(10)])
'6 --> 5. The total went from 22 to 21.'
>>> M = Minimog(type='shuffle')
>>> M.blackjack_move([Integer(0), Integer(2), Integer(4), Integer(6),␣
↪Integer(7), Integer(11)])
'4 --> 3. The total went from 30 to 29.'
```

Is this really a hexad?

```
sage: M.find_hexad([11, 2, 3, 6, 7])
([0, 2, 3, 6, 7, 11], ['square 9', 'picture 1'])
```

```
>>> from sage.all import *
>>> M.find_hexad([Integer(11), Integer(2), Integer(3), Integer(6),␣
↪Integer(7)])
([0, 2, 3, 6, 7, 11], ['square 9', 'picture 1'])
```

So, yes it is, but here is further confirmation:

```
sage: M.blackjack_move([0, 2, 3, 6, 7, 11])
This is a hexad.
There is no winning move, so make a random legal move.
[0, 2, 3, 6, 7, 11]
```

```
>>> from sage.all import *
>>> M.blackjack_move([Integer(0), Integer(2), Integer(3), Integer(6),␣
↪Integer(7), Integer(11)])
This is a hexad.
There is no winning move, so make a random legal move.
[0, 2, 3, 6, 7, 11]
```

Now, suppose player 2 replaced the 11 by a 9. Your next move:

```
sage: M.blackjack_move([0, 2, 3, 6, 7, 9])
'7 --> 1. The total went from 27 to 21.'
```

```
>>> from sage.all import *
>>> M.blackjack_move([Integer(0), Integer(2), Integer(3), Integer(6),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(7), Integer(9)])
'7 --> 1. The total went from 27 to 21.'
```

You have now won. Sage will even tell you so:

```
sage: M.blackjack_move([0, 2, 3, 6, 1, 9])
'No move possible. Shuffle the deck and redeal.'
```

```
>>> from sage.all import *
>>> M.blackjack_move([Integer(0), Integer(2), Integer(3), Integer(6),␣
↪Integer(1), Integer(9)])
'No move possible. Shuffle the deck and redeal.'
```

AUTHOR:

David Joyner (2006-05)

REFERENCES: [CS1986], [KR2001]

find_hexad(*pts*)

Find a hexad of some type.

INPUT:

- `pts` – list S of 5 elements of MINIMOG

OUTPUT: hexad containing $S \cup \{x0\}$ of some type

Note

The 3 "points at infinity" are `{MINIMOG[0][2], MINIMOG[2][1], MINIMOG[0][0]}`.

Theorem ([Cu1984], [Co1984]): Each hexads is of exactly one of the following types:

0. {3 "points at infinity"} union {any line},
1. the union of any two (distinct) parallel lines in the same picture,
2. one "point at infinity" union a cross in the corresponding picture, or
3. two "points at infinity" union a square in the picture corresponding to the omitted point at infinity.

More precisely, there are 132 such hexads (12 of type 0, 12 of type 1, 54 of type 2, and 54 of type 3). They form a Steiner system of type $(5, 6, 12)$.

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: M.find_hexad([0, 1, 2, 3, 4])
([0, 1, 2, 3, 4, 11], ['square 2', 'picture 6'])
sage: M.find_hexad([1, 2, 3, 4, 5])
([1, 2, 3, 4, 5, 6], ['square 8', 'picture 0'])
sage: M.find_hexad([2, 3, 4, 5, 8])
([2, 3, 4, 5, 8, 11], ['lines (1, 2)', 'picture 1'])
sage: M.find_hexad([0, 1, 2, 4, 6])
([0, 1, 2, 4, 6, 8], ['line 1', 'picture 1'])
```

(continues on next page)

(continued from previous page)

```
sage: M = Minimog(type='modulo11')
sage: M.find_hexad([1, 2, 3, 4, SR(infinity)]) # random (machine dependent?)␣
↪order
([+Infinity, 2, 3, 4, 1, 10], ['square 8', 'picture 0'])
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> M.find_hexad([Integer(0), Integer(1), Integer(2), Integer(3), Integer(4)])
([0, 1, 2, 3, 4, 11], ['square 2', 'picture 6'])
>>> M.find_hexad([Integer(1), Integer(2), Integer(3), Integer(4), Integer(5)])
([1, 2, 3, 4, 5, 6], ['square 8', 'picture 0'])
>>> M.find_hexad([Integer(2), Integer(3), Integer(4), Integer(5), Integer(8)])
([2, 3, 4, 5, 8, 11], ['lines (1, 2)', 'picture 1'])
>>> M.find_hexad([Integer(0), Integer(1), Integer(2), Integer(4), Integer(6)])
([0, 1, 2, 4, 6, 8], ['line 1', 'picture 1'])
>>> M = Minimog(type='modulo11')
>>> M.find_hexad([Integer(1), Integer(2), Integer(3), Integer(4),␣
↪SR(infinity)]) # random (machine dependent?) order
([+Infinity, 2, 3, 4, 1, 10], ['square 8', 'picture 0'])
```

AUTHOR:

David Joyner (2006-05)

REFERENCES: [Cu1984], [Co1984]

find_hexad0(*pts*)

Find a hexad of type 0.

INPUT:

- `pts` – set of 2 distinct elements of MINIMOG, but not including the "points at infinity"

OUTPUT:

hexad containing `pts` and of type 0 (the 3 points "at infinity" union a line)

Note

The 3 points "at infinity" are `{MINIMOG[0][2], MINIMOG[2][1], MINIMOG[0][0]}`.

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: M.find_hexad0(set([2, 4]))
([0, 1, 2, 4, 6, 8], ['line 1', 'picture 1'])
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> M.find_hexad0(set([Integer(2), Integer(4)]))
([0, 1, 2, 4, 6, 8], ['line 1', 'picture 1'])
```

find_hexad1 (*pts*)

Find a hexad of type 1.

INPUT:

- `pts` – set of 5 distinct elements of MINIMOG

OUTPUT:

hexad containing `pts` and of type 1 (union of 2 parallel lines – *no* points "at infinity")

> **Note**
>
> The 3 points "at infinity" are `{MINIMOG[0][2], MINIMOG[2][1], MINIMOG[0][0]}`.

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: M.find_hexad1(set([2, 3, 4, 5, 8]))
([2, 3, 4, 5, 8, 11], ['lines (1, 2)', 'picture 1'])
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> M.find_hexad1(set([Integer(2), Integer(3), Integer(4), Integer(5),␣
↪Integer(8)]))
([2, 3, 4, 5, 8, 11], ['lines (1, 2)', 'picture 1'])
```

find_hexad2 (*pts*, *x0*)

Find a hexad of type 2.

INPUT:

- `pts` – list S of 4 elements of MINIMOG, not including any "points at infinity"
- `x0` – in `{MINIMOG[0][2], MINIMOG[2][1], MINIMOG[0][0]}`

OUTPUT: hexad containing $S \cup \{x0\}$ of type 2

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: M.find_hexad2([2, 3, 4, 5], 1)
([], [])
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> M.find_hexad2([Integer(2), Integer(3), Integer(4), Integer(5)],␣
↪Integer(1))
([], [])
```

The above output indicates that there is no hexad of type 2 containing $\{2, 3, 4, 5\}$. However, there is one containing $\{2, 3, 4, 8\}$:

```
sage: M.find_hexad2([2, 3, 4, 8], 0)
([0, 2, 3, 4, 8, 9], ['cross 12', 'picture 0'])
```

```
>>> from sage.all import *
>>> M.find_hexad2([Integer(2), Integer(3), Integer(4), Integer(8)],␣
↪Integer(0))
([0, 2, 3, 4, 8, 9], ['cross 12', 'picture 0'])
```

find_hexad3(*pts*, *x0*, *x1*)

Find a hexad of type 3.

INPUT:

- `pts` – list of 3 elements of MINIMOG, not including any "points at infinity"
- `x0`, `x1` – in `{MINIMOG[0][2], MINIMOG[2][1], MINIMOG[0][0]}`

OUTPUT:

hexad containing pts union {x0, x1} of type 3 (square at picture of "omitted point at infinity")

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: M.find_hexad3([2, 3, 4], 0, 1)
([0, 1, 2, 3, 4, 11], ['square 2', 'picture 6'])
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> M.find_hexad3([Integer(2), Integer(3), Integer(4)], Integer(0),␣
↪Integer(1))
([0, 1, 2, 3, 4, 11], ['square 2', 'picture 6'])
```

print_kitten()

This simply prints the "kitten" (expressed as a triangular diagram of symbols).

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog("shuffle")
sage: M.print_kitten()
         0

         8
       9  10
      5  11  3
     8  2  4  8
   9  10  7  9  10

6                   1
sage: M = Minimog("modulo11")
sage: M.print_kitten()
         +Infinity
```

(continues on next page)

(continued from previous page)

```
          6
        2  10
       5  7  3
      6  9  4  6
    2  10  8  2  10

0                   1
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog("shuffle")
>>> M.print_kitten()
          0
<BLANKLINE>
          8
        9  10
       5  11  3
      8  2  4  8
    9  10  7  9  10
<BLANKLINE>
6                   1
>>> M = Minimog("modulo11")
>>> M.print_kitten()
          +Infinity
<BLANKLINE>
          6
        2  10
       5  7  3
      6  9  4  6
    2  10  8  2  10
<BLANKLINE>
0                   1
```

sage.games.hexad.**picture_set**(*A*, *L*)

This is needed in the *Minimog.find_hexad()* function below.

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: picture_set(M.picture00, M.cross[2])
{5, 7, 8, 9, 10}
sage: picture_set(M.picture02, M.square[7])
{2, 3, 5, 8}
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> picture_set(M.picture00, M.cross[Integer(2)])
{5, 7, 8, 9, 10}
>>> picture_set(M.picture02, M.square[Integer(7)])
```

(continues on next page)

(continued from previous page)

```
{2, 3, 5, 8}
```

sage.games.hexad.**view_list**(L)

This provides a printout of the lines, crosses and squares of the MINIMOG, as in Curtis' paper [Cu1984].

EXAMPLES:

```
sage: from sage.games.hexad import *
sage: M = Minimog(type='shuffle')
sage: view_list(M.line[1])

[0 0 0]
[1 1 1]
[0 0 0]
sage: view_list(M.cross[1])

[1 1 1]
[0 0 1]
[0 0 1]
sage: view_list(M.square[1])

[0 0 0]
[1 1 0]
[1 1 0]
```

```
>>> from sage.all import *
>>> from sage.games.hexad import *
>>> M = Minimog(type='shuffle')
>>> view_list(M.line[Integer(1)])
<BLANKLINE>
[0 0 0]
[1 1 1]
[0 0 0]
>>> view_list(M.cross[Integer(1)])
<BLANKLINE>
[1 1 1]
[0 0 1]
[0 0 1]
>>> view_list(M.square[Integer(1)])
<BLANKLINE>
[0 0 0]
[1 1 0]
[1 1 0]
```

CHAPTER

FOUR

INTERNALS

4.1 This module contains Cython code for a backtracking algorithm to solve Sudoku puzzles.

Once Cython implements closures and the `yield` keyword is possible, this can be moved into the `sage.games.sudoku` module, as part of the `Sudoku.backtrack` method, and this module can be banned.

`sage.games.sudoku_backtrack.`**`backtrack_all`**(*n*, *puzzle*)

A routine to compute all the solutions to a Sudoku puzzle.

INPUT:

- `n` – the size of the puzzle, where the array is an $n^2 \times n^2$ grid
- `puzzle` – list of the entries of the puzzle (1-based), in row-major order

OUTPUT:

A list of solutions, where each solution is a (1-based) list similar to `puzzle`.

ALGORITHM:

We traverse a search tree depth-first. Each level of the tree corresponds to a location in the grid, listed in row-major order. At each location we maintain a list of the symbols which may be used in that location as follows.

A location has "peers", which are the locations in the same row, column or box (sub-grid). As symbols are chosen (or fixed initially) at a location, they become ineligible for use at a peer. We track this in the `available` array where at each location each symbol has a count of how many times it has been made ineligible. As this counter transitions between 0 and 1, the number of eligible symbols at a location is tracked in the `card` array. When the number of eligible symbols at any location becomes 1, then we know that *must* be the symbol employed in that location. This then allows us to further update the eligible symbols at the peers of that location. When the number of the eligible symbols at any location becomes 0, then we know that we can prune the search tree.

So at each new level of the search tree, we propagate as many fixed symbols as we can, placing them into a two-ended queue (`fixed` and `fixed_symbol`) that we process until it is empty or we need to prune. All this recording of ineligible symbols and numbers of eligible symbols has to be unwound as we backup the tree, though.

The notion of propagating singleton cells forward comes from an essay by Peter Norvig [sudoku:norvig].

CHAPTER

FIVE

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

g

sage.games.hexad, 31
sage.games.quantumino, 19
sage.games.sudoku, 3
sage.games.sudoku_backtrack, 41

INDEX

B

backtrack() (*sage.games.sudoku.Sudoku method*), 4
backtrack_all() (*in module sage.games.sudoku_backtrack*), 41
blackjack_move() (*sage.games.hexad.Minimog method*), 32

D

dlx() (*sage.games.sudoku.Sudoku method*), 7

F

find_hexad() (*sage.games.hexad.Minimog method*), 34
find_hexad0() (*sage.games.hexad.Minimog method*), 35
find_hexad1() (*sage.games.hexad.Minimog method*), 35
find_hexad2() (*sage.games.hexad.Minimog method*), 36
find_hexad3() (*sage.games.hexad.Minimog method*), 37

L

list() (*sage.games.quantumino.QuantuminoState method*), 29

M

Minimog (*class in sage.games.hexad*), 32
module
    sage.games.hexad, 31
    sage.games.quantumino, 19
    sage.games.sudoku, 3
    sage.games.sudoku_backtrack, 41

N

number_of_solutions() (*sage.games.quantumino.QuantuminoSolver method*), 23

P

picture_set() (*in module sage.games.hexad*), 38
print_kitten() (*sage.games.hexad.Minimog method*), 37

Q

QuantuminoSolver (*class in sage.games.quantumino*), 23
QuantuminoState (*class in sage.games.quantumino*), 28

S

sage.games.hexad
    module, 31
sage.games.quantumino
    module, 19
sage.games.sudoku
    module, 3
sage.games.sudoku_backtrack
    module, 41
show3d() (*sage.games.quantumino.QuantuminoState method*), 29
show_pentaminos() (*in module sage.games.quantumino*), 30
solve() (*sage.games.quantumino.QuantuminoSolver method*), 24
solve() (*sage.games.sudoku.Sudoku method*), 10
Sudoku (*class in sage.games.sudoku*), 3
sudoku() (*in module sage.games.sudoku*), 16

T

tiling_solver() (*sage.games.quantumino.QuantuminoSolver method*), 27
to_ascii() (*sage.games.sudoku.Sudoku method*), 14
to_latex() (*sage.games.sudoku.Sudoku method*), 14
to_list() (*sage.games.sudoku.Sudoku method*), 15
to_matrix() (*sage.games.sudoku.Sudoku method*), 15
to_string() (*sage.games.sudoku.Sudoku method*), 15

V

view_list() (*in module sage.games.hexad*), 39